

Further Reading

Do not seek to follow in the footsteps of the men of old; seek what they sought.

— Matsuo Basho
Kyoroku Ribetsu no Kotoba, 1693

In this book we have presented a classical method of specifying and verifying sequential programs (Chapters 5 and 6) based on first-order logic (Chapters 1–3) and induction (Chapter 4). We then focused on algorithms for automating the application of this method: decision procedures for reasoning about verification conditions (Chapters 7–11), and invariant generation procedures for deducing inductive facts about programs (Chapter 12). This material is fundamental to all modern research in verification. In this chapter, we indicate topics for further reading and research.

First-Order Logic

Other texts on first-order logic include [87, 31, 55]. Smullyan [87], on which the presentation of Section 2.7 is partly based, concisely presents the main results in first-order logic. Enderton [31] provides a comprehensive discussion of theories of arithmetic and Gödel’s first incompleteness theorem. Manna and Waldinger [55] explore additional first-order theories.

Decision Procedures

We covered three forms of decision procedures: quantifier elimination (Chapter 7) for full theories, decision procedures for quantifier-free fragments of theories with equality (Chapters 8–10), and instantiation-based procedures for limited quantification (Chapter 11). New decision procedures in each of these styles are discovered regularly (see, for example, the proceedings of the *IEEE Symposium on Logic in Computer Science* [51]).

Proofs of correctness in Chapters 7 and 11 appeal to the structure of interpretations. *Model theory* studies logic from the perspective of models (interpretations), and decision procedures can be understood in model-theoretic terms. Hodges covers this topic comprehensively [40].

The combination result of Chapter 10 is not the only one known, although it is the simplest general result. For example, Shostak’s method treats a subclass of fragments with greater speed [84, 78]. Current efforts aim to extend combination methods to require fewer or different restrictions [93, 99, 94, 4]. However, most theories cannot be combined in a general way, particularly when quantification is allowed, so logicians develop special combination procedures [100].

Decision procedures for PL (“SAT solvers”) and combination procedures (“SMT”, for *SAT Modulo Theories*) have received much practical attention recently, even in the form of annual competitions [81, 86]. Motivating applications include software and hardware verification.

Automated Theorem Proving

While satisfiability/validity decision procedures have obvious advantages — speed; the guarantee of an answer in theory and often in practice; and, typically, the ability to produce counterexamples — not all theories or interesting fragments are decidable. For example, reasoning about permutations of arrays is undecidable in certain contexts (see Exercise 6.5 and [77, 6]), yet we would like to prove, for example, that sorting functions return permutations of their input.

Because FOL is complete, semi-decision procedures (see Section 2.6.2) are possible: the procedure described before the proof of Lemma 2.31 is such a procedure. More relevant are widely-used procedures with tactics [76, 67, 47]. Recent work has looked at effectively incorporating decision procedures into general theorem proving [48]. The specification language for verified programming is more expressive using these procedures. However, proving verification conditions sometimes requires user intervention.

FOL may not be sufficiently expressive for some applications. Second-order logic extends first-order logic with quantification over predicates. Despite being incomplete, researchers continue to investigate heuristics for partly automating second-order reasoning [67]. Separation logic is another incomplete logic designed for reasoning about mutable data structures [75].

Static Analysis

Static analysis is one of the most active areas of research in verification. Classically, static analyses of the form presented in Chapter 12 have been studied in two areas: compiler development and research [62] and verification [20].

Important areas of current research include fast numerical analyses for discovering numerical relations among program variables [80], precise alias

and shape analyses for discovering how a program manipulates memory [96], and predicate abstraction and refinement [5, 37, 16, 24].

Static analyses of the form in Chapter 12 solve a set of implications for a fixpoint (an inductive assertion map is a fixpoint) by forward propagation. Other methods exist for finding fixpoints, including constraint-based static analysis [2, 17].

Static analyses also address total correctness by proving that loops and functions halt [18, 7, 8]. Their structure is different than the analyses of Chapter 12 as they seek ranking functions.

Concurrent Programs

We focus on specifying and verifying sequential programs in this book. Just as concurrent programs are more complex than sequential programs, specification and verification methodologies for concurrent programs are more complex than for sequential programs. Fortunately, many of the same methods, including the inductive assertion method and the ranking function method, are still of fundamental importance in concurrent programming. Manna and Pnueli provide a comprehensive introduction to this topic [53, 54]. Milner describes a calculus of concurrent systems in which both the system and its specification is written [60].

Temporal Logic

Because functions of π do not have side effects, specifying function behavior through function preconditions and postconditions is sufficient. However, reactive and concurrent systems, such as operating systems, web servers, and computer processors, exhibit remarkably complex behavior. Temporal logics are typically used for specifying their behavior.

A temporal logic extends PL or FOL with temporal operators that express behavior over time. Canonical behaviors include *invariance*, in which some condition always holds; *progress*, in which a particular event eventually occurs; and *reactivity*, in which a condition causes a particular event to occur eventually.

Temporal logics are divided into logics over linear-time structures (Linear Temporal Logic (LTL)) [53]; branching-time structures (Computational Tree Logic (CTL), CTL*) [14]; and alternating-time structures (Alternating-time Temporal Logic (ATL), ATL*) [3].

Model Checking

A finite-state model checker [15, 74] is an algorithm that checks whether finite-state systems such as hardware circuits satisfy given temporal properties.

An explicit-state model checker manipulates sets of actual states as vectors of bits. A symbolic model checker uses a formulaic representation to represent

sets of states, just as we use FOL to represent possibly infinite sets of states in Chapters 5 and 6. The first symbolic model checker was for CTL [12]; it represents sets of states with Reduced Ordered Binary Decision Diagrams (ROBDDs, or just BDDs) [10, 11].

LTL model checking is based on manipulating automata over infinite strings [95]. A rich literature exists on such automata; see [91] for an introduction.

Predicate abstraction and refinement [5, 37, 16, 24] has allowed model checkers to be applied to software and represents one of the many intersections between areas of research (model checking and static analysis in this case).

Clarke, Grumberg, and Peled discuss model checking in detail [14].

References

1. W. Ackermann. Solvable cases of the decision problem. *The Journal of Symbolic Logic*, 22(1):68–72, March 1957.
2. A. Aiken. Introduction to set constraint-based program analysis. *Science of Computer Programming*, 35(2-3):79–111, November 1999.
3. R. Alur, T. Henzinger, and O. Kupferman. Alternating-time temporal logic. *Journal of the ACM*, 49:672–713, 2002.
4. F. Baader, S. Ghilardi, and C. Tinelli. A new combination procedure for the word problem that generalizes fusion decidability results in modal logics. *Information and Computation*, 204(10):1413–1452, 2006.
5. N. S. Bjørner, A. Browne, and Z. Manna. Automatic generation of invariants and intermediate assertions. *Theoretical Computer Science*, 173(1):49–87, 1997.
6. A. R. Bradley. *Safety Analysis of Systems*. PhD thesis, Stanford University, June 2007.
7. A. R. Bradley, Z. Manna, and H. B. Sipma. Linear ranking with reachability. In *Computer Aided Verification*, volume 3576 of *LNCS*, pages 491–504. Springer-Verlag, 2005.
8. A. R. Bradley, Z. Manna, and H. B. Sipma. Termination of polynomial programs. In *Verification, Model Checking and Abstract Interpretation*, volume 3385 of *LNCS*, pages 113–129. Springer-Verlag, 2005.
9. A. R. Bradley, Z. Manna, and H. B. Sipma. What’s decidable about arrays? In *Verification Model Checking and Abstract Interpretation*, volume 3855 of *LNCS*, pages 427–442. Springer-Verlag, January 2006.
10. R. E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, 35(8):677–691, August 1986.
11. R. E. Bryant. Symbolic Boolean manipulation with ordered binary-decision diagrams. *ACM Computing Surveys*, 24(3):293–318, September 1992.
12. J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill, and L. J. Hwang. Symbolic model checking: 10^{20} states and beyond. *Information and Computation*, 98(2):142–170, 1992.
13. A. Church. A note on the Entscheidungsproblem. *Journal of Symbolic Logic*, 1, 1936.
14. E. Clarke, O. Grumberg, and D. Peled. *Model Checking*. MIT Press, 2000.
15. E. M. Clarke and E. A. Emerson. Design and synthesis of synchronization skeletons using branching-time temporal logic. In *Logic of Programs*, pages 52–71. Springer-Verlag, 1982.

16. E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification*, volume 1855 of *LNCs*, pages 154–169. Springer-Verlag, 2000.
17. M. Colón, S. Sankaranarayanan, and H. B. Sipma. Linear invariant generation using non-linear constraint solving. In *Computer Aided Verification*, volume 2725 of *LNCs*, pages 420–433. Springer-Verlag, 2003.
18. M. Colón and H. B. Sipma. Synthesis of linear ranking functions. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 2031 of *LNCs*, pages 67–81. Springer-Verlag, April 2001.
19. D. C. Cooper. Theorem proving in arithmetic without multiplication. *Machine Intelligence*, 7:91–99, 1972.
20. P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Principles of Programming Languages*, pages 238–252. ACM Press, 1977.
21. P. Cousot and N. Halbwachs. Automatic discovery of linear restraints among the variables of a program. In *Principles of Programming Languages*, pages 84–96. ACM Press, 1978.
22. G. B. Dantzig. *Programming in a Linear Structure*. USAF, February 1948.
23. G. B. Dantzig. Maximization of linear function of variables subject to linear inequalities. In *Cowles Commission for Research in Economics*, volume 13, pages 339–347. Wiley, 1951.
24. S. Das and D. L. Dill. Successive approximation of abstract transition relations. In *Logic in Computer Science*, page 51. IEEE Computer Society, 2001.
25. M. Davis, G. Logemann, and D. Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, July 1962.
26. M. Davis and H. Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7(3):201–215, July 1960.
27. D. L. Detlefs, K. R. M. Leino, G. Nelson, and J. B. Saxe. Extended static checking. Technical Report 159, Compaq SRC, December 1998.
28. E. W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, August 1975.
29. P. J. Downey, R. Sethi, and R. E. Tarjan. Variations on the common subexpressions problem. *Journal of the ACM*, 27(4):758–771, 1980.
30. B. Dreben and H. Putnam. The Craig interpolation lemma. *Notre Dame Journal of Formal Logic*, 8(3):229–233, July 1967.
31. H. B. Enderton. *A Mathematical Introduction to Logic*. Academic Press, 1972.
32. J. Ferrante and C. Rackoff. A decision procedure for the first order theory of real addition with order. *SIAM Journal on Computing*, 4(1):69–76, 1975.
33. M. J. Fischer and M. O. Rabin. Super-exponential complexity of Presburger arithmetic. In R. M. Karp, editor, *Complexity of Computation*, volume 7 of *SIAM-AMS Proceedings*, pages 27–42. American Mathematical Society, 1974.
34. R. W. Floyd. Assigning meanings to programs. In *Symposia in Applied Mathematics*, volume 19, pages 19–32. American Mathematical Society, 1967.
35. K. Gödel. *Über die Vollständigkeit des Logikkalküls*. PhD thesis, University of Vienna, 1929.
36. K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.

37. S. Graf and H. Saidi. Construction of abstract state graphs with PVS. In *Computer Aided Verification*, volume 1254 of *LNCS*, pages 72–83. Springer-Verlag, 1997.
38. D. Hilbert. Die Grundlagen der Mathematik. *Abhandlungen aus dem Seminar der Hamburgischen Universität*, 6:65–85, 1928.
39. C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, October 1969.
40. W. Hodges. *A Shorter Model Theory*. Cambridge University Press, 1997.
41. J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Automata Theory, Languages, and Computation*. Addison-Wesley, 3rd edition, 2006.
42. R. A. Horn and C. R. Johnson. *Matrix Analysis*. Cambridge University Press, 1985.
43. J. Jaffar. Presburger arithmetic with array segments. *Information Processing Letters*, 12(2):79–82, 1981.
44. L. Kantorovich. Mathematical methods of organizing and planning production. *Management Science*, 6:366–422, 1960. In Russian, Leningrad University, 1939.
45. N. Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4:373–395, 1984.
46. M. Karr. Affine relationships among variables of a program. *Acta Informatica*, 6:133–151, 1976.
47. M. Kaufmann, P. Manolios, and J. S. Moore. *Computer-Aided Reasoning: An Approach*. Kluwer Academic Publishers, 2000.
48. M. Kaufmann, J. S. Moore, S. Ray, and E. Reeber. Integrating external deduction tools with ACL2. In *Workshop on the Implementation of Logics*, volume 212, pages 7–26, 2006.
49. L. G. Khachian. A polynomial algorithm in linear programming. *Soviet Math. Dokl.*, 20:191–194, 1979. In Russian, *Dokl. Akad. Nauk SSSR* 244, 1093–1096, 1979.
50. J. King. *A Program Verifier*. PhD thesis, Carnegie Mellon University, September 1969.
51. IEEE Symposium on Logic in Computer Science. <http://www2.informatik.hu-berlin.de/lics>.
52. Z. Manna. *Mathematical Theory of Computation*. McGraw-Hill, 1974. Also Dover, 2004.
53. Z. Manna and A. Pnueli. *The Temporal Logic of Reactive and Concurrent Systems: Specification*. Springer-Verlag, 1991.
54. Z. Manna and A. Pnueli. *Temporal Verification of Reactive Systems: Safety*. Springer-Verlag, 1995.
55. Z. Manna and R. Waldinger. *The Deductive Foundations of Computer Programming*. Addison-Wesley, 1993.
56. Z. Manna and C. G. Zarba. Combining decision procedures. In *Formal Methods at the Cross Roads: From Panacea to Foundational Support*, volume 2757 of *LNCS*, pages 381–422. Springer-Verlag, 2003.
57. P. Mateti. A decision procedure for the correctness of a class of programs. *Journal of the ACM*, 28(2), 1981.
58. J. McCarthy. Towards a mathematical science of computation. In *International Federation for Information Processing*, pages 21–28, 1962.
59. J. McCarthy. A basis for a mathematical theory of computation. *Computer Programming and Formal Systems*, 1963.

60. R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.
61. A. Miné. The octagon abstract domain. In *Analysis, Slicing and Transformation (part of Working Conference on Reverse Engineering)*, IEEE, pages 310–319. IEEE Computer Society, October 2001.
62. S. S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann Publishers Inc., 1997.
63. M. Müller-Olm and H. Seidl. A note on Karr’s algorithm. In *International Colloquium on Automata, Languages and Programming*, volume 3142 of *LNCS*, pages 1016–1027. Springer-Verlag, 2004.
64. G. Nelson. Combining satisfiability procedures by equality-sharing. *Contemporary Mathematics*, 29:201–211, 1984.
65. G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM Transactions on Programming Languages and Systems*, 1(2):245–257, October 1979.
66. G. Nelson and D. C. Oppen. Fast decision procedures based on congruence closure. *Journal of the ACM*, 27(2):356–364, April 1980.
67. T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *LNCS*. Springer-Verlag, 2002.
68. D. C. Oppen. A $2^{2^{2^{pn}}}$ upper bound on the complexity of Presburger arithmetic. *Journal of Computer and System Sciences*, 16(3):323–332, 1978.
69. D. C. Oppen. Reasoning about recursively defined data structures. In *Principles of Programming Languages*, pages 151–157. ACM Press, 1978.
70. D. C. Oppen. Complexity, convexity and combinations of theories. *Theoretical Computer Science*, 12:291–302, 1980.
71. D. C. Oppen. Reasoning about recursively defined data structures. *Journal of the ACM*, 27(3):403–411, July 1980.
72. C. H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1993.
73. M. Presburger. Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchen die Addition als einzige Operation hervortritt. In *Comptes Rendus du Premier Congrès des Mathématiciens des Pays Slaves*, pages 92–101, 1929.
74. J. P. Queille and J. Sifakis. Specification and verification of concurrent systems in CESAR. In *International Symposium on Programming*, volume 137 of *LNCS*. Springer-Verlag, 1982.
75. J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Logic in Computer Science*, pages 55–74. IEEE Computer Society, 2002.
76. A. Riazanov and A. Voronkov. The design and implementation of VAMPIRE. *AI Communications*, 15(2):91–110, September 2002.
77. L. E. Rosier. A note on Presburger arithmetic with array segments, permutation and equality. *Information Processing Letters*, 22(1):33–35, 1986.
78. H. Ruess and N. Shankar. Deconstructing Shostak. In *Logic in Computer Science*. IEEE Computer Society, 2001.
79. B. Russell. *Philosophy of Logical Atomism*, 1918.
80. S. Sankaranarayanan, M. Colón, H. Sipma, and Z. Manna. Efficient strongly relational polyhedral analysis. In *Verification, Model Checking, and Abstract Interpretation*, volume 3855 of *LNCS*, pages 111–125, 2006.
81. The international SAT competitions web page. <http://www.satcompetition.org>.
82. A. Schrijver. *Theory of Linear and Integer Programming*. Wiley, 1986.

83. R. E. Shostak. An algorithm for reasoning about equality. *Communications of the ACM*, 21(7):583–585, July 1978.
84. R. E. Shostak. Deciding combinations of theories. *Journal of the ACM*, 31(1):1–12, January 1984.
85. M. Sipser. *Introduction to the Theory of Computation*. Course Technology, 1996.
86. SMT-LIB: SMT solver competition. <http://goedel.cs.uiowa.edu/smtlib>.
87. R. M. Smullyan. *First-Order Logic*. Dover, 1968.
88. A. Stump, C. W. Barrett, D. L. Dill, and J. Levitt. A decision procedure for an extensional theory of arrays. In *Logic in Computer Science*, pages 29–37. IEEE Computer Society, 2001.
89. N. Suzuki and D. Jefferson. Verification decidability of Presburger array programs. *Journal of the ACM*, 27(1):191–205, January 1980.
90. A. Tarski. *A Decision Method for Elementary Algebra and Geometry*. University of California Press, 1951.
91. W. Thomas. Automata on infinite objects. In J. van Leeuwen, editor, *Handbook of Theoretical Computer Science*, volume B, pages 133–191. Elsevier North-Holland, Inc., 1990.
92. C. Tinelli and M. T. Harandi. A new correctness proof of the Nelson-Oppen combination procedure. In *Frontiers of Combining Systems*, pages 103–120. Kluwer Academic Publishers, 1996.
93. C. Tinelli and C. Ringeissen. Unions of non-disjoint theories and combinations of satisfiability procedures. *Theoretical Computer Science*, 290(1):291–353, January 2003.
94. C. Tinelli and C. Zarba. Combining non-stably infinite theories. Technical Report TinZar-RR-03, University of Iowa, 2003.
95. M. Y. Vardi. An automata-theoretic approach to linear temporal logic. In *Logics for Concurrency: Structure versus Automata*, pages 238–266. Springer-Verlag, 1996.
96. R. Wilhelm, S. Sagiv, and T. W. Reps. Parametric shape analysis via 3-valued logic. *Transactions on Programming Languages and Systems*, 24(3):217–298, May 2002.
97. M. Yadegari. The use of mathematical induction by Abu Kamil Shuja' Ibn Aslam (850-930). *Isis*, 69(2):259–262, June 1978.
98. R. Zach. Hilbert's program. In E. N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. The Metaphysics Research Lab, Fall 2003. <http://plato.stanford.edu/archives/fall2003/entries/hilbert-program>.
99. C. Zarba, Z. Manna, and H. B. Sipma. Combining theories sharing dense orders. In *Automated Reasoning with Analytic Tableaux and Related Methods: Position Papers and Tutorials*, pages 83–98, 2003.
100. T. Zhang, H. B. Sipma, and Z. Manna. Decision procedures for term algebras with integer constraints. *Information and Computation*, 204(10):1526–1574, October 2006.

Index

- + associativity [axiom] 80, 81, 83
- + commutativity [axiom] 80, 81, 83
- + identity [axiom] 80, 81, 83
- + inverse [axiom] 80, 81, 83
- + ordered [axiom] 81, 83
- associativity [axiom] 80, 81
- commutativity [axiom] 80, 81
- identity [axiom] 81
- inverse [axiom] 81
- left identity [axiom] 80
- ordered [axiom] 81
- right identity [axiom] 80
- NPTIME 54
- NPTIME-complete 55
- NP 54
- NP-hard 55
- PTIME 54
- P 54

- abelian group, theory of 80, 83
- abs [program] 173
- absorption laws 342
- abstract domain 321, 343
 - interval 321
 - Karr's 321
- abstract forward propagation algorithm 324
- abstract interpretation 312, 321
- abstract strongest postcondition 322
- ABSTRACTFORWARDPROPAGATE
 - [function] 324
- abstraction function 343
- ack left zero [axiom] 106
- ack right zero [axiom] 106
- ack successor [axiom] 106
- adjacent vertices 218
- admits (quantifier elimination) 184
- affine assertion 333
- affine combination 334
- affine hull 336
- affine space 333
- affine transformation 335
- algorithm 54
- ancestor (semantic argument) 12
- annotation 118
 - assertion 122
 - function call assertion 131
 - function postcondition 118
 - function precondition 118
 - loop invariant 121
 - runtime assertion 122
- antecedent 4
- antisymmetric 341
- antisymmetry [axiom] 81, 82
- application programming interface 175
- arguments 115
- arity 4
 - binary 4
 - unary 4
- arrangement 273
- array congruence [axiom] 88, 263
- array predicate
 - bounded equality 168
 - partitioned 127
 - sorted 120

- weak permutation 170, 174
 - array property 292, 300
 - array property fragment 292, 300
 - arrays, theory of 87, 291
 - ascending chain condition 344
 - assertion 122
 - assertion map 316
 - assignment (interpretation) 39
 - assignment statement 126
 - associative 341
 - assume statement 126
 - at least one root [axiom] 81, 82
 - atom 4, 36
 - atom [axiom] 85, 86, 259
 - augmented matrix 211
 - axiom schema 71
 - axioms 69
 - + associativity 80, 81, 83
 - + commutativity 80, 81, 83
 - + identity 80, 81, 83
 - + inverse 80, 81, 83
 - + ordered 81, 83
 - associativity 80, 81
 - commutativity 80, 81
 - identity 81
 - inverse 81
 - left identity 80
 - ordered 81
 - right identity 80
 - ack left zero 106
 - ack right zero 106
 - ack successor 106
 - antisymmetry 81, 82
 - array congruence 88, 263
 - at least one root 81, 82
 - atom 85, 86, 259
 - concat. atom 98
 - concat. list 98
 - construction 85, 86, 259
 - distributivity 81
 - divides 187
 - divisible 83
 - exp_3 successor 96
 - exp_3 zero 96
 - exp. successor 96
 - exp. zero 96
 - extensionality 89, 293
 - flat atom 98
 - flat list 98
 - function congruence 71, 242
 - hashtable congruence 305
 - induction 73, 75
 - keys-put 305
 - keys-remove 305
 - left distributivity 80
 - left projection 85, 259
 - length atom 102
 - length list 102
 - plus successor 73, 75
 - plus zero 73, 75
 - predicate congruence 71, 243
 - projection 86
 - quotient less 100
 - quotient successor 100
 - read-over-put 1 305
 - read-over-put 2 305
 - read-over-remove 305
 - read-over-write 1 88, 263
 - read-over-write 2 88, 263
 - reflexivity 71, 242, 305
 - remainder less 100
 - remainder successor 100
 - reverse atom 98
 - reverse list 98
 - right distributivity 80
 - right projection 85, 259
 - separate identities 81
 - square-root 81
 - successor 73, 75
 - symmetry 71, 242, 305
 - times successor 73
 - times zero 73
 - torsion-free 83
 - totality 81, 82
 - transitivity 71, 81, 82, 242, 305
 - two 270
 - zero 73, 75
-
- base case 96, 98
 - basic path 125, 316
 - basis 211
 - BCP 28
 - Bell numbers 275
 - binary 4
 - BinarySearch [program] 115–176
 - Boolean connectives 4
 - bound variable 36
 - bounded equality 168

- branch (semantic argument) 12
- BubbleSort [program] 116–176
- canonical form 325
- cardinality 39
- CCPAR [function] 255
- characteristic formula 63
- class
 - congruence 245
 - equivalence 245
- clause 21
- closed (semantic argument) 12
- closed formula 37
- CNF *see* conjunctive normal form
- co-NP 55
- commutative 341
- Compactness Theorem 61
- complete 70
- complete (lattice) 342
- complete (proof method) 56
- complete (theory) 70
- complete induction 95, 99
- complete induction principle 100
- composition (substitution) 18
- computation 142
- concat. atom [axiom] 98
- concat. list [axiom] 98
- concrete domain 343
- concrete semantics 343
- concretization function 343
- congruence class 245
 - representative 252
- congruence closure 244, 247
 - algorithm 248
 - parents 253
- congruence closure parents 253
- congruence relation 72, 243, 245
- congruence-closure algorithm 241
- CONGRUENT [function] 255
- conjunctive fragment 208
- conjunctive normal form 21
- consequent 4
- consistent 70
- constant 35
- constraint 206
- constraint representation (affine space)
 - 334
- constraint representation (polyhedron)
 - 238
- construction [axiom] 85, 86, 259
- constructor 259
- contrapositive 89
- convex 276
- convex hull 239
- convex space 218, 238
- convex theory 276
- Cooper's method 187
 - divisibility predicate 186
 - left elimination 194
 - left infinite projection 189
 - periodicity property 189
 - right elimination 194
 - right infinite projection 194
 - triangular form 197
- Craig Interpolation Lemma 61, 284
- cutpoint 316
- cutset 316
- cycle (graph) 251
- cylindrical algebraic decomposition 82
- DAG *see* directed acyclic graph
- De Morgan's Law 19
- decidable 54, 70
- decision procedure 21
- deduction (semantic argument) 10
- descendant (semantic argument) 12
- dimension 211
- direct descendant (semantic argument)
 - 12
- directed acyclic graph 251
- disjoint (partition) 246
- disjunctive normal form 20
- distributivity [axiom] 81
- divides [axiom] 187
- divisibility predicate 186
- divisible [axiom] 83
- DNF *see* disjunctive normal form
- domain (interpretation) 39
- domain (substitution) 16
- DPLL decision procedure 28
 - Boolean constraint propagation 28
 - unit clause 29
 - unit resolution 29
- dual optimization problem 215
- echelon form 212
- edge (graph) 251
- elementarily equivalent 83

- elementary algebra *see* reals, theory of
- elementary recursive 90
- elementary row operations 211
- entry function 142
- equality with uninterpreted functions
 - see* equality, theory of
- equality, theory of 71, 241
- equationally satisfied 215
- equisatisfiable 24
- equivalence class 245
- equivalence closure 247
- equivalence relation 71, 242, 245
- equivalent 14, 70
 - weakly 284
- EUF *see* equality, theory of
- existential closure 37
- existential quantifier 36
- exp*₃ successor [axiom] 96
- exp*₃ zero [axiom] 96
- exp. successor [axiom] 96
- exp. zero [axiom] 96
- extensionality 85, 293
- extensionality [axiom] 89, 293
- falsifying interpretation 10
- Ferrante and Rackoff's method 200
 - left infinite projection 201
 - right infinite projection 201
- field, theory of 80
- FIND [function] 254
- finished (semantic argument) 13
- first-order arithmetic *see* Peano arithmetic
- first-order logic 4, 35
 - atom 36
 - constant 35
 - formula 36
 - closed 37
 - existential closure 37
 - universal closure 37
 - function 36
 - interpretation 39
 - assignment 39
 - domain 39
 - variant 41
 - literal 36
 - predicate 36
 - propositional variable 36
 - quantifier 36
 - bound variable 36
 - existential 36
 - quantified variable 36
 - scope 36
 - universal 36
- satisfiable 42
- semantics 39
- subformula 37
 - strict 37
- substitution 46
- subterm 37
 - strict 38
- term 35
- valid 42
- variable 35
 - bound 37
 - free 37
- first-order theory *see* theory
- fixpoint 342
- flat atom [axiom] 98
- flat list [axiom] 98
- FOL *see* first-order logic
- formal parameters 115
- formula 4, 36, 69
 - closed 37
 - existential closure 37
 - universal closure 37
- formula schema 48
 - placeholder 48
 - side condition 48
 - valid 49
- formula template 16
- forward propagation 312, 317
- FORWARDPROPAGATE [function] 318
- fragment 70
 - array property 292, 300
 - conjunctive 90, 208
 - hashtable property 292, 305
 - pure equality 284
 - quantifier-free 70
- free variable 37
- function 36
- function call assertion 131
- function congruence [axiom] 71, 242
- function postcondition 118
- function precondition 118
- functions
 - ABSTRACTFORWARDPROPAGATE 324

- CCPAR 255
- CONGRUENT 255
- FIND 254
- FORWARDPROPAGATE 318
- MERGE 256
- NODE 254
- UNION 254
- WIDEN 324

- Gaussian elimination 211
 - echelon form 212
 - elementary row operations 211
- global optimum 218
- graph 251
 - directed 251
 - directed acyclic 251
 - edge 251
 - node 251
- graph coloring 33
- greatest fixpoint 343
- group
 - torsion-free, theory of 83
- group, theory of 80

- has equality 284
- hashtable congruence [axiom] 305
- hashtable property 305
- hashtable property fragment 292, 305
- hashtables, theory of 291
- Hintikka set 60
- Hintikka's Lemma 60
- Hoare triple 137
- hull
 - affine 336
 - convex 239
 - interval 326

- idempotent 341
- identity matrix 210
- implies 15
 - weakly 284
- index guard 292, 300
- index set 295
- induction 95
 - complete 95, 99
 - lexicographic well-founded 104
 - stepwise 95, 96, 98
 - structural 60, 95, 108
 - well-founded 95, 102

- induction [axiom] 73, 75
- inductive 143
- inductive assertion map 317
- inductive assertion method 113, 124, 143
- inductive definition 7
- inductive hypothesis 96, 98, 100, 103, 109
- inductive map *see* inductive assertion map
- inductive step 96, 98
- infimum *see* meet
- InsertionSort [program] 150, 174, 176
- integer-indexed arrays, theory of 300
- integers, theory of 73, 76, 183
- intended interpretation 74
- interpolant 61
- interpretation 6, 39, 70
 - assignment 39
 - domain 39
 - falsifying 10
 - intended 74, 75, 77
 - variant 41
- interpreted 71
- intersection [program] 175, 176
- interval abstract domain 321
- interval analysis 311
- interval arithmetic 326
- interval hull 326
- invariant 143
- invariant generation 311
- invertible matrix 213
- irreflexive 112

- join 341

- König's Lemma 59
- Karr's abstract domain 321
- Karr's analysis 311, 333
- key guard 305
- key set 304
- keys-put [axiom] 305
- keys-remove [axiom] 305

- Löwenheim's Theorem 60
- Löwenheim-Skolem Theorem 61
- lattice 341
 - complete 342
- least fixpoint 343

- left distributivity [axiom] 80
- left elimination 194
- left infinite projection
 - Cooper's method 189
 - Ferrante and Rackoff's method 201
- left projection [axiom] 85, 259
- left projector 259
- length atom [axiom] 102
- length function 142
- length list [axiom] 102
- lexicographic relation 103
- lexicographic well-founded induction 104
- line (semantic argument) 12
- linear equation 211
- linear inequality 214
- LinearSearch [program] 115–176
- lists, theory of 84
- literal 4, 36
- local optimum 218
- logical connectives 4
- loop invariant 121
- matrix 209
 - augmented 211
 - identity 210
 - invertible 213
 - nonsingular 213
 - square 211
 - triangular form 211
- matrix-matrix multiplication 210
- matrix-vector multiplication 210
- meet 341
- MERGE [function] 256
- merge [program] 174, 177
- MergeSort [program] 174
- minimal disjunction 283
- modus ponens 13
- monotone function 342
- ms [program] 174, 177
- N-O *see* Nelson-Oppen combination method
 - negation normal form 19
 - Nelson-Oppen combination method 269, 270
 - arrangement 273
 - NNF *see* negation normal form
 - NODE [function] 254
 - node (graph) 251
 - nondeterministic-polynomial time 54
 - nonsingular matrix 213
 - normal form 18
 - conjunctive 21
 - disjunctive 20
 - negation 19
 - prenex 52
 - null space 213
 - objective function 206, 214
 - open (semantic argument) 13
 - optimization problem 206, 214
 - constraint 206
 - constraints 214
 - dual 215, 216
 - objective function 206, 214
 - primal 216
 - order 55
 - order (group) 83
 - ordered field, theory of 81
 - parent (semantic argument) 12
 - partial correctness 113
 - partial order 341
 - partially correct 124
 - partially decidable *see* semi-decidable
 - partially ordered set 341
 - partition 246
 - partition [program] 164–176
 - partitioned 127
 - path 124
 - basic 125, 316
 - Peano arithmetic, theory of 73, 96
 - periodicity property 189
 - pivot rule 236
 - PL *see* propositional logic
 - placeholder (formula schema) 48
 - plus successor [axiom] 73, 75
 - plus zero [axiom] 73, 75
 - PNF *see* prenex normal form
 - polyhedra
 - vertex enumeration algorithm 239
 - polyhedron 214, 238
 - constraint representation 238
 - ray 238
 - vertex representation 238
 - polynomial time 54
 - poset *see* partially ordered set

- precondition method 156
- predicate 36
- predicate calculus *see* first-order logic
- predicate congruence** [axiom] 71, 243
- predicate logic *see* first-order logic
- predicate transformer 136, 312
 - strongest postcondition 312–316
 - weakest precondition 136, 312–316
- premise (semantic argument) 10
- prenex normal form 52
- Presburger arithmetic, theory of 73, 75
- primed variables 315
- program 142
- program annotation 113
- program counter 135, 315
- programs
 - abs 173
 - BinarySearch 115–176
 - BubbleSort 116–176
 - InsertionSort 150, 174, 176
 - intersection 175, 176
 - LinearSearch 115–176
 - merge 174, 177
 - MergeSort 174
 - ms 174, 177
 - partition 164–176
 - qsort 164–176
 - QuickSort 164–176
 - random 164–176
 - subset 175, 176
 - union 175, 178, 179
- progress property 113
- projection** [axiom] 86
- projector 259
- proof rules (semantic argument) 10
- proof tactics 22
- propositional calculus *see* propositional logic
- propositional logic 4
 - atom 4
 - Boolean connectives 4
 - DPLL decision procedure 28
 - Boolean constraint propagation 28
 - unit clause 29
 - unit resolution 29
 - equivalent 14
 - formula 4
 - implies 15
 - interpretation 6
 - falsifying 10
 - literal 4
 - logical connectives 4
 - resolution
 - resolvent 27
 - resolution decision procedure 27
 - satisfiable 8
 - semantic argument method 10
 - semantics 6
 - subformula 5
 - strict 5
 - substitution 16
 - syntax 4
 - truth symbols 4
 - truth table 6
 - truth values 6
 - truth-table method 9
 - valid 8
 - variables 4
- propositional variable 36
- pure equality fragment 284
- QE *see* quantifier elimination
- qsort** [program] 164–176
- quantifier 36
 - bound variable 36
 - existential 36
 - quantified variable 36
 - scope 36
 - universal 36
- quantifier alternation 208, 291
- quantifier elimination 82, 183, 184
 - admits 184
 - integer arithmetic *see* Cooper's method
 - rational arithmetic *see* Ferrante and Rackoff's method
- quantifier elimination procedure 183
- quantifier instantiation 291
- quantifier-free fragment 70, 208
- QuickSort** [program] 164–176
- quotient 246
- quotient less** [axiom] 100
- quotient successor** [axiom] 100
- random [program] 164–176
- range (substitution) 16
- ranking function 114, 144

- ranking function method 114
- rational, theory of 79, 82, 183, 207
- RDS *see* recursive data structures, theory of
- reachable state 321
- read-over-put 1 [axiom] 305
- read-over-put 2 [axiom] 305
- read-over-remove [axiom] 305
- read-over-write 1 [axiom] 88, 263
- read-over-write 2 [axiom] 88, 263
- real closed field, theory of 81
- reals, theory of 79, 80
- recursive *see* decidable, recursive
- recursive data structures, theory of 84
- recursively enumerable *see* semi-decidable
- refinement (relation) 246
- reflexive 245, 341
- reflexivity [axiom] 71, 242, 305
- relation
 - congruence 72, 243, 245
 - equivalence 71, 242, 245
 - lexicographic 103
 - well-founded 102
- remainder less [axiom] 100
- remainder successor [axiom] 100
- renaming (substitution) 46
- representative (congruence class) 252
- resolution
 - resolvent 27
- resolution decision procedure 27
- resolvent 27
- reverse atom [axiom] 98
- reverse list [axiom] 98
- right distributivity [axiom] 80
- right elimination 194
- right infinite projection
 - Cooper's method 194
 - Ferrante and Rackoff's method 201
- right projection [axiom] 85, 259
- right projector 259
- ring, theory of 80
- runtime assertion 122
- runtime error 122

- safe (substitution) 47
- safety property 113
- satisfiable 8, 42, 70
- schema (substitution) 48

- scope 36
- semantic argument method 10
 - ancestor 12
 - branch 12
 - closed 12
 - deduction 10
 - descendant 12
 - direct descendant 12
 - finished 13
 - line 12
 - open 13
 - parent 12
 - premise 10
 - proof rules 10
 - derived 13
- semantics 6, 39
- semi-decidable 54
- separate identities** [axiom] 81
- side condition (formula schema) 48
- signature 69
- simplex method 207, 218
- simultaneously satisfiable 61
- sorted 120
- sound 56
- source (edge) 251
- space
 - affine 333
 - vector 210
- specification 113
- square matrix 211
- square-root** [axiom] 81
- stably infinite 270
- state 135
- statement
 - assignment 126
 - assume 126
- static analysis 311
 - interval analysis 311, 325
 - Karr's analysis 311, 333
- stepwise induction 95, 96, 98
- stepwise induction principle 96, 98
- strengthened hypothesis 97
- strict subformula 5
- strict subformula relation 108
- strict subterms 38
- strongest postcondition 312–316
 - abstract 322
- structural induction 95, 108
- structural induction principle 109

- subformula 5, 37
 - strict 5, 37
- subformula ordering 8
- subset** [program] 175, 176
- substitution 16, 46
 - composition 18
 - domain 16
 - range 16
 - renaming 46
 - safe 47
 - schema 48
 - variable 17
- subterm 37
 - set 247
 - strict 38
- subterm set 247
- successor** [axiom] 73, 75
- successor location 318
- supremum *see* join
- symbolic execution 317
- symmetric 245
- symmetry** [axiom] 71, 242, 305
- syntax 4

- target (edge) 251
- term 35
- theory 69
 - axioms 69
 - complete 70
 - consistent 70
 - convex 276
 - decidable 70
 - equivalent 70
 - formula 69
 - fragment 70
 - conjunctive 90
 - quantifier-free 70
 - has equality 284
 - interpretation 70
 - intended 74, 75, 77
 - satisfiable 70
 - signature 69
 - stably infinite 270
 - valid 70
- theory of
 - abelian group 80, 83
 - arrays 87, 291
 - equality 71, 241
 - field 80
 - group 80
 - torsion-free 83
 - hashtables 291
 - integer-indexed arrays 300
 - integers 73, 76, 183
 - lists 84
 - ordered field 81
 - Peano arithmetic 73, 96
 - Presburger arithmetic 73, 75
 - rational numbers 79, 82, 183, 207
 - real closed field 81
 - reals 79, 80
 - recursive data structures 84
 - ring 80
 - total order 81, 83
 - theory of rationals 79
 - theory of reals 79
 - times successor** [axiom] 73
 - times zero** [axiom] 73
 - torsion-free 83
 - torsion-free** [axiom] 83
 - total (partition) 246
 - total correctness 113, 143
 - total order, theory of 81, 83
 - totality** [axiom] 81, 82
 - transition relation 315
 - transitive 245, 341
 - transitivity** [axiom] 71, 81, 82, 242, 305
 - transpose 209
 - triangular form (Cooper's method) 197
 - triangular form (matrix) 211
 - truth symbols 4
 - truth table 6
 - truth values 6
 - truth-table method 9
 - Turing machine 54
 - Turing-decidable *see* decidable
 - Turing-recognizable *see* semi-decidable
 - two** [axiom] 270

 - unary 4
 - undecidable 54
 - UNION** [function] 254
 - union** [program] 175, 178, 179
 - union-find algorithm 254
 - unit clause 29
 - unit resolution 29

- unit vector 210
- universal closure 37
- universal quantifier 36
- unsatisfiable core 237
- valid 8, 42, 49, 70
- valid (formula schema) 49
- valid abstraction 344
- value constraint 292, 300, 305
- variable 35
 - bound 37
 - free 37
- variable (substitution) 17
- variable vector 209
- variables 4
- variant interpretation 41
- VC *see* verification condition
- vector 209
 - unit 210
- vector space 210
 - basis 211
 - dimension 211
- vector-vector multiplication 209
- verification condition 124, 136, 137
- verifying compiler 114
- vertex 214
 - defining constraints 214
- vertex enumeration algorithm 239
- vertex representation (affine space) 334
- vertex representation (polyhedron) 238
- weak permutation 170, 174
- weakest precondition 136, 312–316
- weakly equivalent 284
- weakly implies 284
- well-founded induction 95, 102
- well-founded induction principle 103
- well-founded relation 102
- WIDEN [function] 324
- widening 321
- widening operator 323
- zero [axiom] 73, 75